

1 Plan qualité

Stratégie de tests par couche, conventions, workflow Git, CI/CD et onboarding : de quoi produire des livrables fiables et intégrer un nouveau développeur rapidement.

1.1 Definition of Done

Conditions qu’une User Story doit toutes remplir pour être considérée comme terminée :

- Les critères d’acceptation (Given/When/Then) sont tous satisfaits.
- Le code est relu et approuvé par au moins un pair via une Pull Request.
- Les tests de la story sont écrits et passent : **unitaires et fonctionnels obligatoires** (intégration et E2E si la story touche un flux critique).
- La couverture de code atteint le seuil fixé (mesurée en CI, merge bloqué sinon).
- La CI est verte : format, lint, tests et audit automatiques réussis (le merge est bloqué sinon).
- La documentation utile est à jour (README du service, doc d’API, Confluence si besoin).
- Aucune régression : un bug corrigé est couvert par un test de non-régression.

RÈGLE D'OR PRODUIT

L’IA n’émet jamais de diagnostic. Les tests du Care Engine vérifient que toute situation ambiguë déclenche une orientation ou une escalade vers le vétérinaire, jamais une conclusion médicale.

1.2 Stratégie de tests par couche

Deux niveaux sont **obligatoires** pour chaque story (inscrits dans la Definition of Done) : tests **unitaires** (logique isolée) et **fonctionnels** (comportement d’un module ou d’un endpoint). Les **flux critiques** sont en plus couverts par des tests **E2E / smoke** : onboarding et appairage, ingestion jusqu’à l’alerte, escalade IA vers vétérinaire, paiement. La couverture est mesurée et contrôlée en CI (seuil ci-dessous).

Couche	Outils	Cible
Firmware collier (C / Zephyr)	Tests unitaires (Zephyr) + analyse statique (Clang-Tidy, Cppcheck, MISRA C) + simulateur de capteurs	Logique de collecte/encodage validée hors matériel ; sûreté mémoire par MISRA C + allocation statique.
Services backend (Rust)	Tests unitaires (cargo nextest) + tests d'intégration API sur services réels (testcontainers)	Logique métier et endpoints couverts ; tests d'abus obligatoires sur l'authentification.
Moteur IA / Care Engine (Python)	pytest + jeux d'évaluation RAG + garde-fous anti-diagnostic	Non-régression des réponses et escalade systématique en cas de doute.
App mobile (Kotlin / SwiftUI)	Tests unitaires natifs + tests d'UI critiques	Onboarding, appairage et alertes vérifiés.
Bout en bout (E2E / smoke)	Scénarios end-to-end sur les flux critiques (phase d'intégration)	Collier → backend → IA → app/portail validés ensemble.
Performance / charge	Tests de charge (k6 / Locust) sur les endpoints critiques	Temps de réponse p95 et montée en charge validés avant la mise en production (NFR6, NFR7).

1.3 Couverture de code : objectif & seuil

Chaque story livre ses tests ; la **couverture de ligne** est mesurée en CI et **bloque le merge** sous le seuil. Nous nous appuyons sur les paliers de Google ([Code Coverage Best Practices](#)) : **60 % « acceptable », 75 % « commendable », 90 % « exemplary »**.

- **Objectif : 90 %** (palier « exemplary ») sur le **code métier critique** : garde-fous du Care Engine, ingestion, authentification, paiement.
- **Plancher bloquant en CI : 80 %** sur l'ensemble du code (via `cargo llvm-cov --fail-under-lines`), au-dessus du palier « commendable ».
- **Pas de course au 100 %** : au-delà de 90 %, le gain devient logarithmique (beaucoup d'effort pour un risque résiduel faible) ; Google déconseille explicitement les objectifs projet au-dessus de 90 %.
- Ce qui compte autant que le chiffre : l'**analyse humaine des lignes non couvertes** (le risque résiduel est-il acceptable ?), pas la métrique brute.

1.4 Conventions de code

- Linting/formatage automatiques par langage : rustfmt + clippy (Rust), ruff/black (Python), ktlint (Kotlin), ESLint + Prettier (TS/Next).
- Nommage cohérent par langage (snake_case Rust/Python, camelCase Kotlin/TS) et noms descriptifs.
- Toute fonctionnalité passe par une Pull Request relue par au moins un pair avant merge.
- Pas de secret en clair dans le code : variables d'environnement + secrets chiffrés (SOPS).

1.5 Workflow Git

Branche	Rôle
main	Branche stable et déployable. Protégée : merge uniquement via PR validée.
develop	Intégration continue des fonctionnalités avant stabilisation.
feat/<scope>	Une fonctionnalité = une branche, rattachée à un ticket Jira.
fix/<scope>	Correction de bug isolée et traçable.

Commits conventionnels (feat:, fix:, docs:, chore:...) référençant la clé Jira. **Pull Request** : description claire, lien Jira, CI verte et revue d'au moins un pair avant merge.

1.6 Intégration continue (CI)

Architecture quasi **full-Rust** : la CI (GitHub Actions) est centrée sur l'outillage Cargo. À chaque Pull Request, **tout gate est bloquant** (merge impossible si rouge) :

Étape	Commande / rôle
Format	<code>cargo fmt --all --check</code> : style uniforme, non négociable.
Tri des dépendances	<code>cargo sort --workspace --check</code> : Cargo.toml ordonnés.
Lint	<code>cargo clippy --workspace --all-targets -- -D warnings</code> : zéro warning toléré.
Dépendances inutilisées	<code>cargo machete</code> : pas de dette de dépendances.
Tests + couverture	<code>cargo llvm-cov nextest --workspace --fail-under-lines 80</code> : tests rapides (nextest) + seuil de couverture.
Audit de sécurité	<code>cargo audit</code> : CVE des dépendances (base RustSec).

Les tests d'intégration tournent sur des **services réels éphémères** (bases, brokers) via testcontainers. Les autres stacks ont leurs propres gates : **Python** (Care Engine) ruff + pytest + jeux d'évaluation ;

mobile SwiftLint / ktlint (--strict) + tests natifs avec couverture. Un cache (Swatinem/rust-cache) accélère la CI. La branche `main` est **protégée** : merge uniquement via PR verte et relue.

1.7 Sécurité des dépendances & supply chain

- **Dependabot** : mises à jour hebdomadaires et groupées (crates Cargo, GitHub Actions, dépendances mobile), ouvertes en PR automatiques passant par la même CI.
- **cargo audit** (RustSec) à chaque PR + job de sécurité dédié ; **cargo deny** prévu pour le contrôle des licences.
- **Secrets chiffrés** (SOPS + age), jamais en clair dans le dépôt ; scan de secrets sur les PR.
- **Protection de branche** : revue d'au moins un pair obligatoire, historique linéaire, CI verte requise.

1.8 Déploiement (CD)

- Build et publication d'**images conteneurisées** (Docker) versionnées à chaque release.
- **Trois environnements** : `develop` → **staging** (validation intégrée) → **production**, promus par merge ; un canal **hotfix** dédié pour les correctifs urgents en production.
- Déploiement **GitOps** : Helm + Argo CD synchronisent le cluster Kubernetes depuis Git (source de vérité).
- Observabilité OpenTelemetry → Prometheus / Loki / Tempo → Grafana ; rollback par simple revert Git.

1.9 Onboarding développeur

- Accès dépôt GitHub, projet Jira et espace Confluence dès l'arrivée.
- Lecture d'entrée : ce plan qualité, le Git Workflow et l'architecture globale.
- Mise en route locale : clone, installation, lancement en local (README de chaque service).
- Première contribution : un ticket « bonne première tâche », une PR suivant les conventions.